Tabla de contenidos

1. I/O de archivos
2. *Context Manager*
3. (Bonus) Cómo emular archivos para I/O

I/O de archivos

Hasta ahora, en IIC1103, han operado con la lectura y escritura de archivos de texto; sin embargo, los sistemas operativos representan los archivos como secuencias de *bytes*, no como texto. Dado que leer *bytes* y convertirlos a texto es una operación muy común en archivos, los lenguajes de programación proveen maneras de manejar los *bytes* para transformarlos a una representación en *string* usando métodos de codificación (*encoding*) y decodificación (*decoding*).

La función `open` nos permite, además de abrir archivos, ingresar como argumentos el set de caracteres que se usará para codificar los *bytes* y la estrategia que se debe seguir cuando aparezcan *bytes* inconsistentes con el formato:

```
# Creamos un archivo con un texto de base.
contenido = "¿Qué pasa con las tildes? " \
    "¿Por qué no aparecen los signos de interrogación del comienzo? :
("

# Guardamos el contenido en un archivo con encoding UTF-8
file = open("data/IO_archivo_ejemplo", "w", encoding='utf-8')
file.write(contenido)

# Pero leemos el mismo archivo con encoding ASCII
file = open('data/IO_archivo_ejemplo', "r", encoding='ascii',
errors='replace')
print(file.read())
file.close()
```

```
��Qu�� pasa con las tildes? ��Por qu�� no aparecen los signos de
interrogaci��n del comienzo? :(
```

En este caso, se añadió el argumento `errors="replace"` para indicar a la función `open` cómo debe manejar algún error del archivo cuando es incapaz de entender algún caracter. Por defecto, debería ocurrir un error, pero usando `"replace"` hacemos que no se caiga el programa y solo reemplace el caracter que no puede leer por el símbolo `�`.

Ahora, veamos cómo cambia esto, si es que elegimos otro *encoding* para leer el contenido del archivo.

```
file = open('data/IO_archivo_ejemplo', "r", encoding='utf-8')
print(file.read())
file.close()
```

```
¿Qué pasa con las tildes? ¿Por qué no aparecen los signos de
interrogación del comienzo? :(
```

Vemos que ahora sí se muestran bien las tildes y los signos de interrogación, pues al leer el archivo lo estamos haciendo con el mismo *encoding* con el que fue escrito. Ahora, escribiremos en el mismo archivo, un texto distinto, y luego leeremos el archivo para ver qué ocurre.

```
contenido = "sorry pero ahora yo soy lo que habrá dentro del archivo"

file = open("data/IO_archivo_ejemplo", "w", encoding="utf-8")
file.write(contenido)
file.close()

file = open('data/IO_archivo_ejemplo', "r", encoding='utf-8')
print(file.read())
file.close()
```

```
sorry pero ahora yo soy lo que habrá dentro del archivo
```

Como puedes haber notado, el archivo se sobrescribió, y el nuevo texto en lugar de ser agregado al archivo, reemplazó lo que había. Para agregar un nuevo texto al final de un archivo ya existente, debemos cambiar el modo de apertura del archivo cambiando la `w` de *write* por una `a`, de *append*. De esta forma, al escribir en el archivo, se va a hacer al final del archivo en lugar de reemplazar el contenido anterior.

```
contenido = "\nyo me agregaré al final"

# Abrimos el archivo en modo append
file = open("data/IO_archivo_ejemplo", "a", encoding="utf-8")
file.write(contenido)
file.close()

# Abrimos el archivo en el modo de lectura (read)
file = open('data/IO_archivo_ejemplo', "r", encoding='utf-8')
print(file.read())
file.close()
```

```
sorry pero ahora yo soy lo que habrá dentro del archivo
yo me agregaré al final
```

Además de leer archivos de texto, podemos abrir archivos y leer sus *bytes* en lugar de texto. Para abrir un archivo como binario, simplemente debemos agregar una `b` por el lado derecho del modo de apertura. Por ejemplo, `wb` (*write bytes*) o `rb` (*read bytes*). El archivo se comportará igual que un archivo de texto, sólo que sin la codificación automática de *byte* a texto.

```
contenido = b"abcde12"

file = open("data/IO_archivo_ejemplo_2", "wb")
file.write(contenido)
```

```
file.close()

file = open('data/IO_archivo_ejemplo_2', "rb")
print(file.read())
file.close()
```

```
b'abcde12'
```

Podemos concatenar *bytes* simplemente con el operador `+`. En el siguiente ejemplo, construimos un contenido dinámico para ser escrito en un archivo de *bytes*. Después leemos una cantidad fija de *bytes* desde el mismo archivo:

```
num_lineas = 100

file = open("data/IO_archivo_ejemplo_3", "wb")
for i in range(num_lineas):
    # La función bytes() necesita una lista de números que se puedan
recorrer uno a uno,
    # por eso usamos [i] en lugar de solo i.
    contenido = b"linea_" + bytes([i]) + b" abcde12 "
    file.write(contenido)
file.close()

file = open('data/IO_archivo_ejemplo_3', "rb")
# El número dentro de la función read nos dice el número de bytes que
# se van a leer del archivo
print(file.read(41))
file.close()
```

```
b'linea_\x00 abcde12 linea_\x01 abcde12 linea_\x02 a'
```

Vamos a utilizar este modo de abrir archivos como *bytes* más adelante en el curso

Context Manager

Dado que siempre necesitamos cerrar un archivo después de usarlo, debemos considerar la posibilidad de que ocurran excepciones mientras el archivo está abierto.

Una forma de hacerlo es utilizar un bloque del tipo `try/finally` para cerrar el archivo. Este tipo de bloque primero ejecutará todo el contenido dentro del `try`. Luego, aunque haya un error en el código, ejecutará lo que esté dentro del bloque `finally`.

Sin embargo, esto genera bastante código extra y requiere entender bien qué hace el bloque `try/finally` (lo cual vamos a aprender con detalle más adelante). En Python existe una forma de hacer lo mismo con menos código, a través de un *context manager*, que se encarga de ejecutar, por debajo, las sentencias `try` y `finally` sin la necesidad de llamarlas directamente. Sólo necesitamos llamar al archivo que queremos abrir usando la sentencia `with`. Veamos un ejemplo.

```
with open("data/IO_archivo_ejemplo_3", "r") as file:
    contenido = file.read()
```

El código anterior sería equivalente a hacer lo siguiente:

```
file = open("data/IO_archivo_ejemplo_3", "r")
try:
    contenido = file.read()
finally:
    file.close()
```

Ahora, si ejecutamos `dir` en un objeto de tipo archivo:

```
file = open("data/IO_archivo_ejemplo_3", "w")
print(dir(file))
file.close()
```

```
['_CHUNK_SIZE', '__class__', '__del__', '__delattr__', '__dict__',
'__dir__', '__doc__', '__enter__', '__eq__', '__exit__', '__format__',
'__ge__', '__getattribute__', '__getstate__', '__gt__', '__hash__',
'__init__', '__init_subclass__', '__iter__', '__le__', '__lt__',
'__ne__', '__new__', '__next__', '__reduce__', '__reduce_ex__',
'__repr__', '__setattr__', '__sizeof__', '__str__',
'__subclasshook__', '_checkClosed', '_checkReadable',
'_checkSeekable', '_checkWritable', '_finalizing', 'buffer', 'close',
'closed', 'detach', 'encoding', 'errors', 'fileno', 'flush', 'isatty',
'line_buffering', 'mode', 'name', 'newlines', 'read', 'readable',
'readline', 'readlines', 'reconfigure', 'seek', 'seekable', 'tell',
'truncate', 'writable', 'write', 'write_through', 'writelines']
```

Vemos que existen dos métodos llamados `__enter__` y `__exit__`. Estos dos métodos transforman el archivo en un *context manager*. El método `__exit__` asegura que el archivo será cerrado incluso si aparece una excepción mientras esté abierto. El método `__enter__` inicializa el archivo o realiza cualquier acción necesaria para ajustar el contexto del objeto.

Para asegurarnos que un archivo usará los métodos `__enter__` y `__exit__`, simplemente debemos llamar a la apertura del archivo con el método `with`.

También podemos crear nuestros propios *context managers* a partir de cualquier clase. Simplemente agregamos los métodos `__enter__` y `__exit__` y podemos llamar a nuestra clase a través del método `with`. Del siguiente ejemplo, se puede ver cómo el método `__exit__` se ejecuta una vez que nos salimos del *scope* de la sentencia `with`.

```
import string
import random


class StringUpper:
```

```
    def __init__(self):
        print("1. Inicizalizando context manager")
        self.data = []

    def __enter__(self):
        print("2. Abriendo context manager y definiendo qué elemento
será el `as XXXX`")
        return self.data

    def __exit__(self, exc_type, exc_value, traceback):
        print("5. Cerrando context manager")
        i = 0
        for char in self.data:
            self.data[i] = char.upper()
            i+= 1



with StringUpper() as s_upper:
    # s_upper será una lista según lo definido en "__enter__"
    print("3. Primera línea dentro del context manager")

    for i in range(20):
        # Aquí seleccionamos, en forma aleatoria, un ascii en
minúsculas
        # y las agregamos a la lista
        s_upper.append(random.choice(string.ascii_lowercase))

    print("\t" + str(s_upper))
    print("4. Última línea del context manager")

print("\t" + str(s_upper))
```

```
1. Inicizalizando context manager
2. Abriendo context manager y definiendo qué elemento será el `as
XXXX`
3. Primera línea dentro del context manager
     ['f', 'j', 'e', 'r', 's', 'n', 'l', 'k', 'n', 'q', 's', 'g', 'r',
'z', 'd', 'l', 't', 'm', 'i', 't']
4. Última línea del context manager
5. Cerrando context manager
     ['F', 'J', 'E', 'R', 'S', 'N', 'L', 'K', 'N', 'Q', 'S', 'G', 'R',
'Z', 'D', 'L', 'T', 'M', 'I', 'T']
```

El código anterior simplemente corresponde a una clase que implementar los métodos `__enter__` y `__exit__`. Con esto, podemos instanciar la clase a través de un *context manager*. En este ejemplo en particular, el *context manager* se encarga de transformar todos los caracteres ASCII de la lista a mayúsculas cuando este es cerrado.

(Bonus) Cómo emular archivos para I/O

Muchas veces tenemos que interactuar con algunos módulos de *software* que sólo leen y escriben sus datos desde y hacia archivos. Si queremos comunicar nuestro código que genera -por ejemplo, *strings*- para evitar tener que escribir nuestros datos en un archivo para que el otro programa los lea, podemos *emular* el tener un archivo usando los módulos de Python `StringIO` o `BytesIO`. El siguiente ejemplo muestra cómo usar estos módulos:

```
from io import StringIO, BytesIO


# Aquí simulamos tener un archivo que contiene el string dado
file_in = StringIO("información como texto y más")

# Aquí simulamos un archivo de Bytes para escribir la información
file_out = BytesIO()

char = file_in.read(1)
while char:
    file_out.write(char.encode("ascii", "ignore"))
    char = file_in.read(1)

buffer_ = file_out.getvalue()
print(buffer_)
```

```
b'informacin como texto y ms'
```